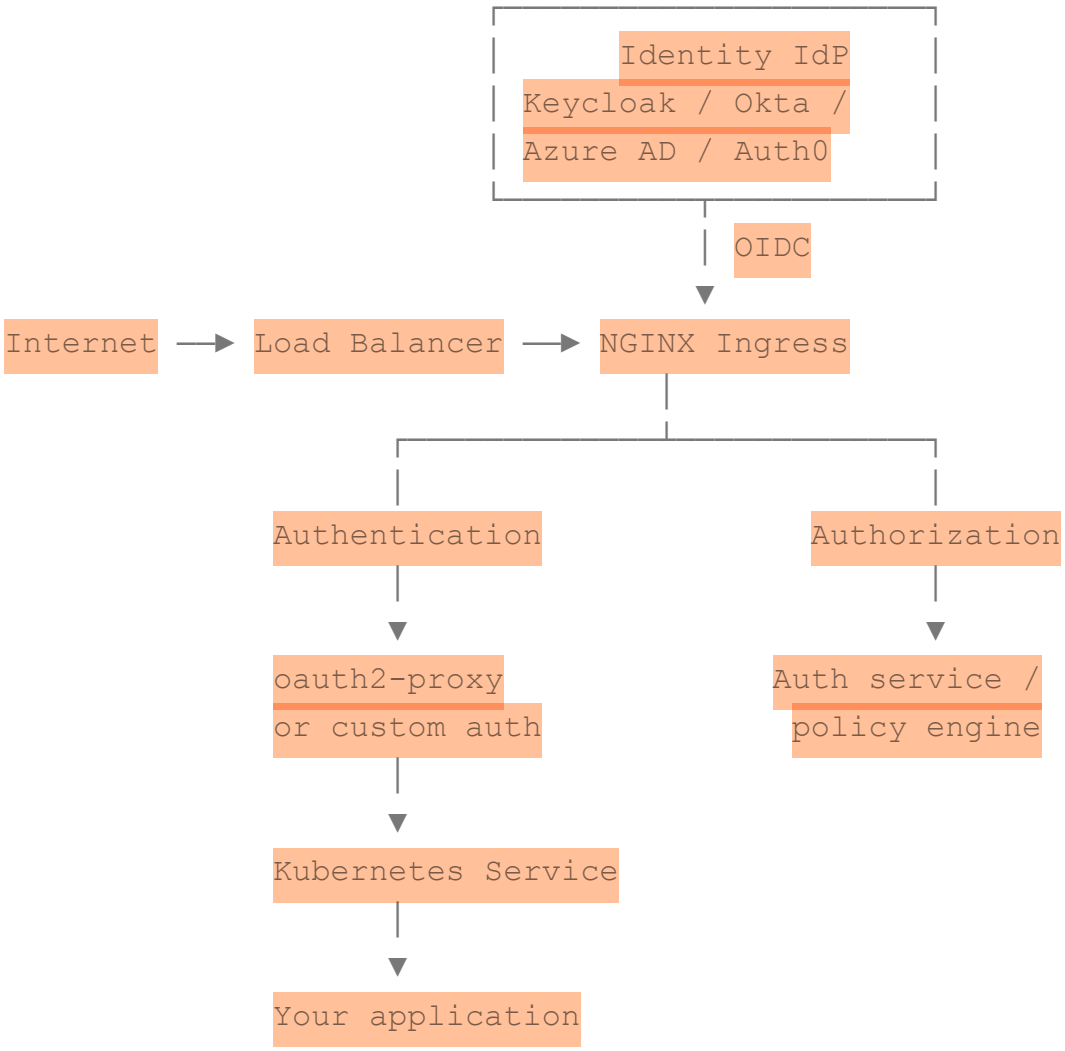
For Kubernetes, the cleanest way to configure **NGINX Ingress Controller authentication and authorization** is to separate the concerns:



The important distinction is:

- **Authentication (AuthN)** → *Who are you?*
- **Authorization (AuthZ)** → *Are you allowed to perform this operation?*

NGINX Ingress can perform the authentication gate using its `auth-url` / `auth-signin` functionality, delegating the actual identity verification to an external authentication service. Kubernetes

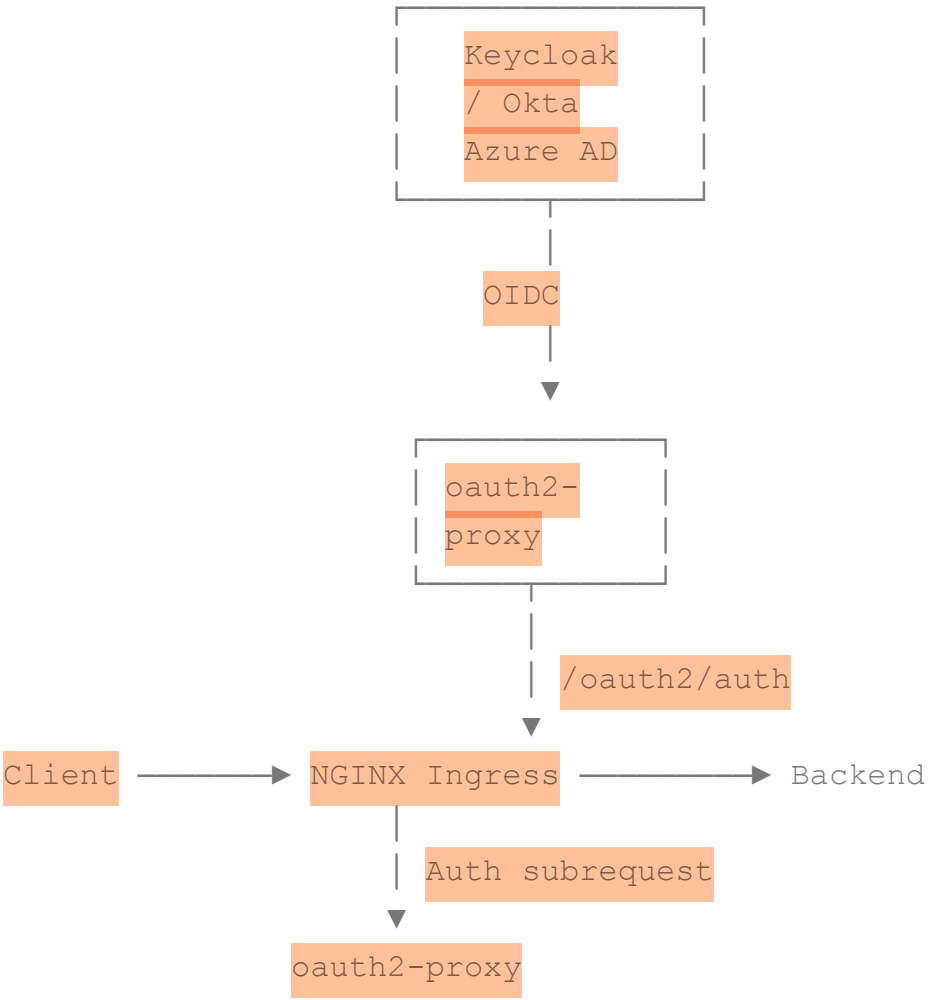
---

1. Recommended architecture

For a production microservices platform, I would recommend:

OIDC Provider → OAuth2 Proxy → NGINX Ingress → Application

For example:



This is preferable to implementing JWT/OIDC logic directly inside NGINX annotations.

The official ingress-nginx documentation provides this exact OAuth2 Proxy pattern using `auth-url` and `auth-signin`. Kubernetes

---

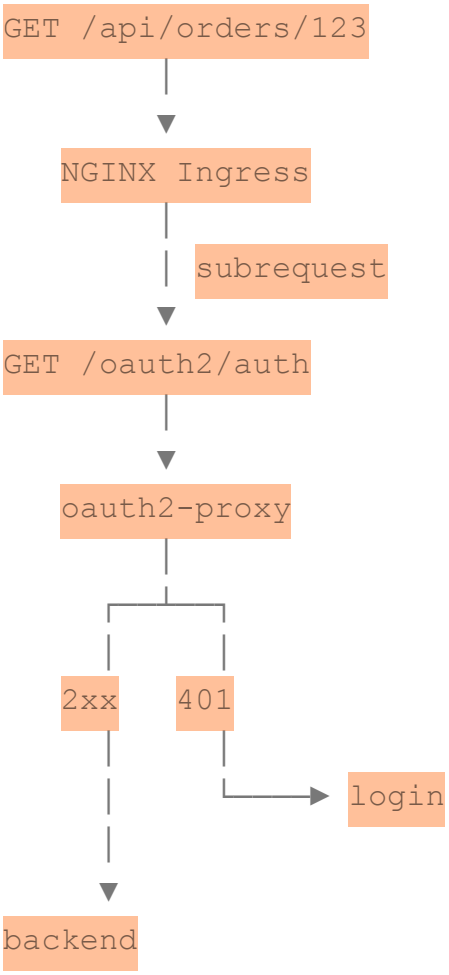
2. Authentication with `auth-url`

The key annotation is:

```
nginx.ingress.kubernetes.io/auth-url: "https://$host/oauth2/auth"
```

NGINX makes an internal authentication request before forwarding the original request to your application. GitHub

Conceptually:



A successful authentication response allows the original request to continue.

---

3. OAuth2 Proxy Ingress

For example:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: oauth2-proxy
  namespace: security
spec:
  ingressClassName: nginx

  rules:
    - host: app.example.com
      http:
        paths:
          - path: /oauth2
```

```
            pathType: Prefix
            backend:
              service:
                name: oauth2-proxy
                port:
                  number: 4180

  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
```

OAuth2 Proxy handles:

- OIDC authorization code flow
- login redirects
- ID/access tokens
- session cookies
- token validation
- interaction with your IdP

The ingress-nginx OAuth example uses the same general architecture. GitHub

---

4. Protect your application

Then create a separate Ingress for the actual application:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orders-api
  namespace: application

  annotations:
    nginx.ingress.kubernetes.io/auth-url: "https://$host/oauth2/auth"
    nginx.ingress.kubernetes.io/auth-signin: "https://$host/oauth2/start?

spec:
  ingressClassName: nginx

  rules:
```

```
    - host: app.example.com
      http:
        paths:
          - path: /api/orders
            pathType: Prefix
            backend:
              service:
                name: orders-service
                port:
                  number: 8080

  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
```

The important pieces are:

```
nginx.ingress.kubernetes.io/auth-url: "https://$host/oauth2/auth"

nginx.ingress.kubernetes.io/auth-signin: "https://$host/oauth2/start?rd=$
```

The second annotation tells NGINX where to redirect an unauthenticated browser request. Kubernetes

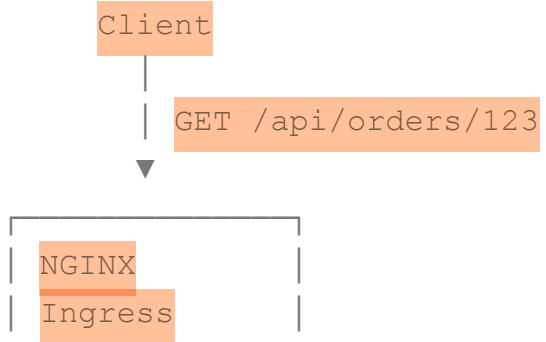
---

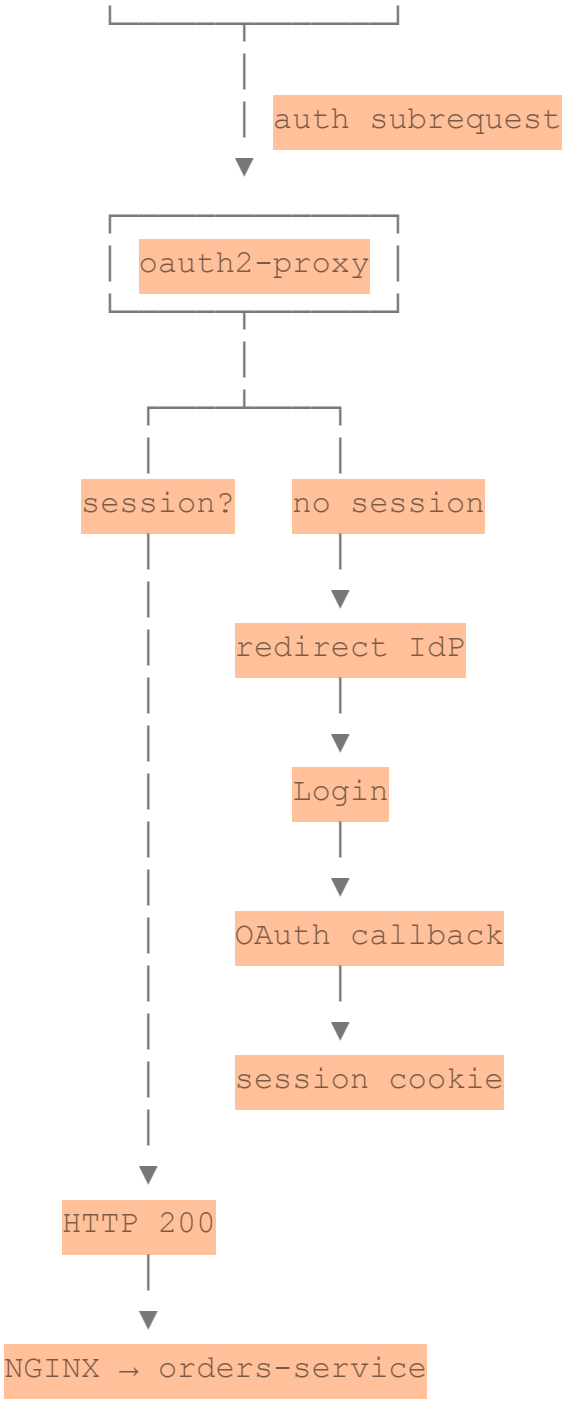
5. Authentication flow

Suppose the user requests:

```
GET https://app.example.com/api/orders/123
```

NGINX effectively performs:





---

6. Passing identity to the backend

Authentication alone isn't enough.

Your backend often needs to know:

```
user ID
username
email
groups
roles
```

```
tenant
claims
```

NGINX Ingress supports propagating selected headers returned by the authentication service using:

```
nginx.ingress.kubernetes.io/auth-response-headers: "X-Auth-Request-User,X
```

This is explicitly supported by ingress-nginx's external authentication functionality. Kubernetes

For example:

```
metadata:
  annotations:

    nginx.ingress.kubernetes.io/auth-url: "https://$host/oauth2/auth"

    nginx.ingress.kubernetes.io/auth-signin: "https://$host/oauth2/start?

    nginx.ingress.kubernetes.io/auth-response-headers: >-
      X-Auth-Request-User,
      X-Auth-Request-Email,
      X-Auth-Request-Groups
```

Your application could then receive:

```
X-Auth-Request-User: 12345
X-Auth-Request-Email: user@example.com
X-Auth-Request-Groups: engineers,admins
```

---

7. Authorization

This is where architecture becomes important.

I would **not** try to encode complex authorization logic directly into NGINX.

For example, avoid turning Ingress annotations into something like:

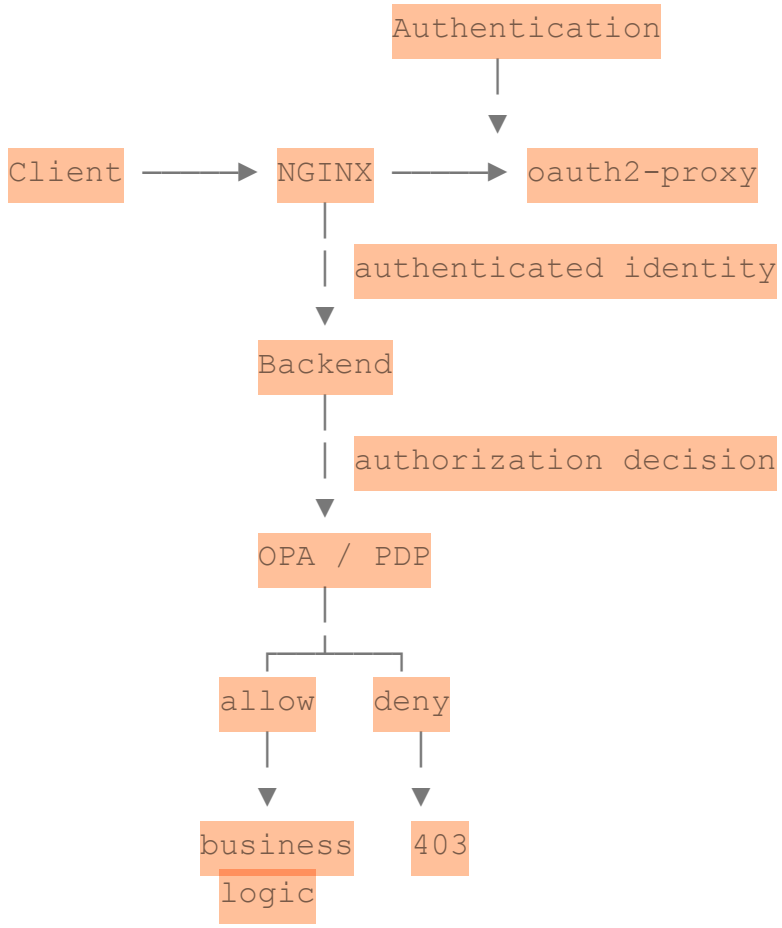
```
/admin → role=ADMIN
/orders → role=USER
```

```
/orders/{id} → owner OR ADMIN
/payments → FINANCE
```

NGINX is excellent as an **authentication enforcement point**, but your authorization policy should normally live in:

- your application
- a dedicated authorization service
- or a policy engine such as **Open Policy Agent (OPA)**

A good architecture is:



---

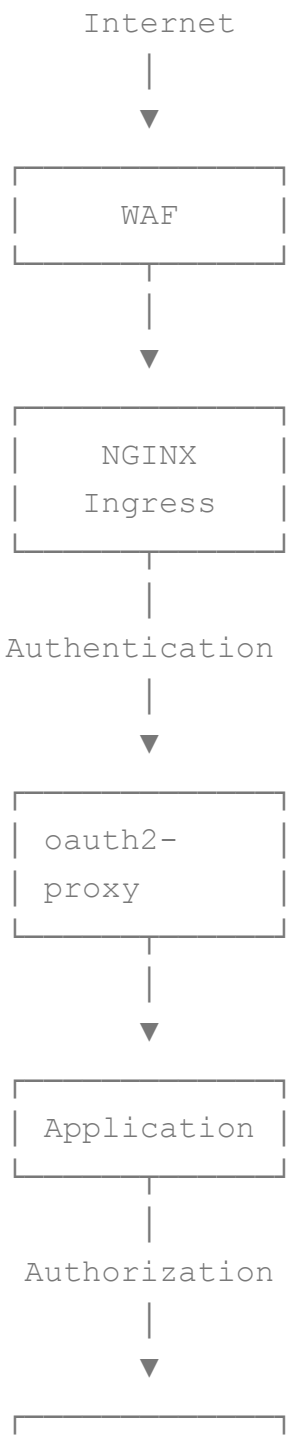
8. Authentication vs authorization layers

I recommend using multiple layers:

Layer	Responsibility
Load Balancer	Network exposure
NGINX	TLS termination, routing, coarse access control

Layer	Responsibility
OAuth2 Proxy	Authentication/session
IdP	Identity
OPA/Policy Engine	Authorization
Application	Business authorization
Database	Data-level isolation

For example:



```
                    | OPA / PDP    |
                    └──────────────┘
```

---

9. API authentication is slightly different

For a browser application:

```
Browser
   |
   | session cookie
   ▼
NGINX → OAuth2 Proxy → Application
```

For a REST API consumed by another service:

```
Client
   |
   | Authorization: Bearer <JWT>
   ▼
NGINX
   |
   ▼
Authentication service
   |
   ▼
Application
```

You don't necessarily want to redirect an API client to a login page.

Instead:

```
GET /api/orders

Authorization: Bearer eyJ...
```

should result in:

```
valid token ──────► 200
invalid token ────► 401
```

```
valid identity
but forbidden ⟶ 403
```

rather than:

```
401 → browser redirect
```

So I would usually have different authentication policies for:

```
/api/**
```

versus:

```
/web/**
```

---

10. Basic authentication

NGINX Ingress also supports native Basic/Digest authentication.

For example:

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/auth-type: "basic"
    nginx.ingress.kubernetes.io/auth-secret: "basic-auth"
    nginx.ingress.kubernetes.io/auth-realm: "Authentication Required"
```

The Secret contains an `htpasswd` file.

The official ingress-nginx documentation supports both `auth-file` and `auth-map` forms. [Kubernetes]

This is appropriate for things such as:

```
internal admin endpoint
temporary staging environment
simple internal service
CI endpoint
```

but I wouldn't use Basic Auth as the primary identity system for a large enterprise platform.

---

11. Mutual TLS authentication

For service-to-service communication, another option is mTLS.

NGINX Ingress supports client certificate authentication through annotations such as:

```
nginx.ingress.kubernetes.io/auth-tls-secret: "security/client-ca"
nginx.ingress.kubernetes.io/auth-tls-verify-client: "on"
nginx.ingress.kubernetes.io/auth-tls-verify-depth: "2"
```

This is useful for:

```
Service A
   │
   │ mTLS
   ▼
NGINX
   │
   ▼
Service B
```

However, in a Kubernetes service mesh architecture, I'd normally let the mesh handle workload identity/mTLS and reserve NGINX for north-south traffic.

---

12. Global authentication

If **almost every application** behind the ingress requires authentication, you can configure global external authentication.

The ingress-nginx ConfigMap supports:

```
global-auth-url
global-auth-signin
global-auth-response-headers
```

so authentication doesn't need to be repeated on every Ingress. Kubernetes

Conceptually:

```
NGINX Controller
      │
      │ global auth
      ▼
oauth2-proxy
      │
      ├── application A
      ├── application B
      ├── application C
      └── application D
```

Individual routes can opt out using:

```
nginx.ingress.kubernetes.io/enable-global-auth: "false"
``` :chatgpt-content-reference{index="8"}


I'd use this carefully because public endpoints such as:

```text
/health
/metrics
/webhooks
/oauth2/*
/.well-known/*
```

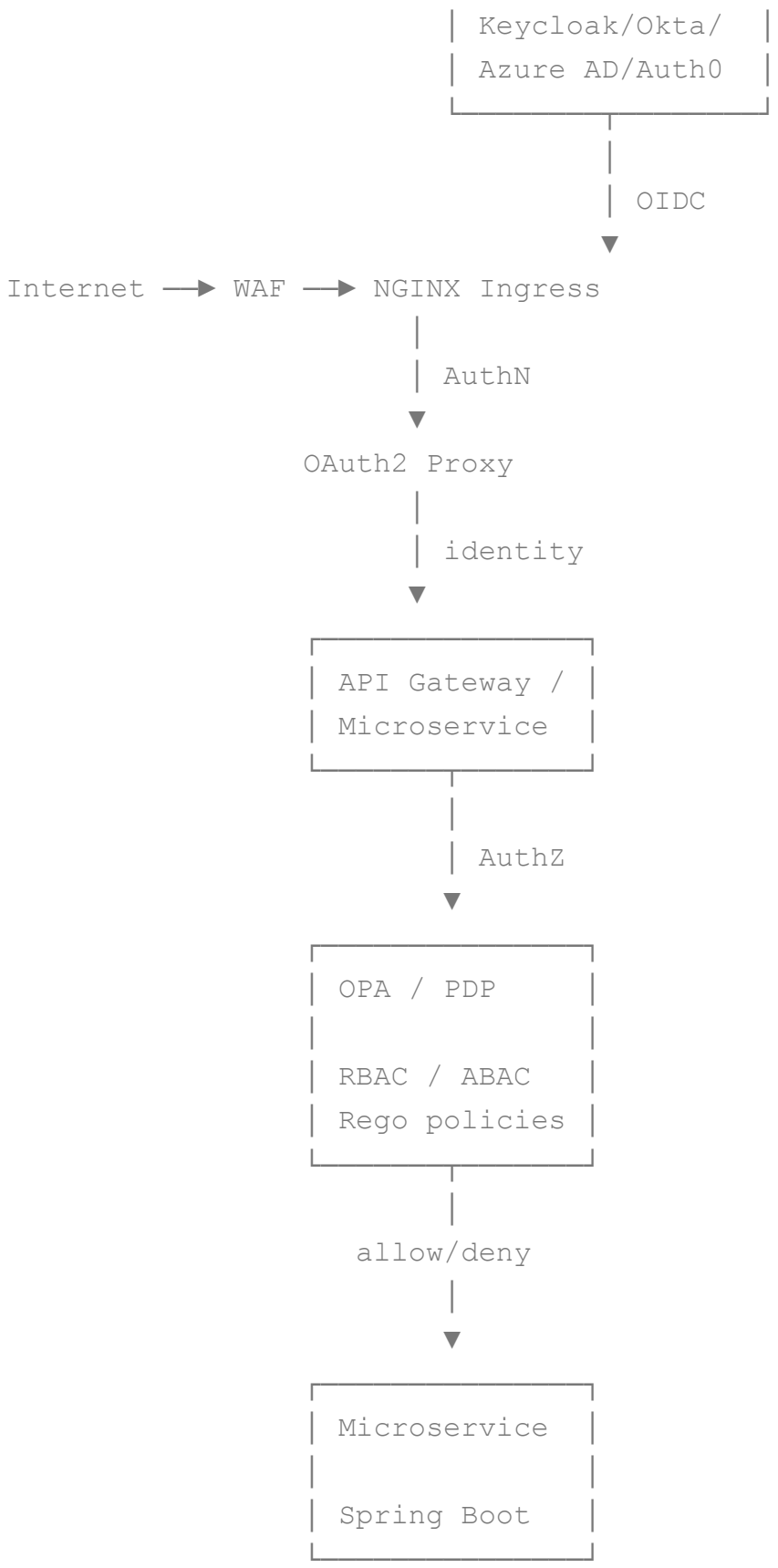
may need different policies.

---

13. A production-grade design

For the kind of cloud-native architecture you're working toward, I'd use something closer to:

```
                    ┌──────────────────┐
                    │ OIDC Identity    │
                    │ Provider         │
                    │                  │
```



The application should **still perform authorization checks** for business-sensitive operations.

For example:

```
GET /orders/123
```

may require:

```
authenticated
AND
user has ORDER_READ
AND
user belongs to tenant 42
```

while:

```
DELETE /orders/123
```

might require:

```
authenticated
AND
user has ORDER_DELETE
AND
user owns order
OR
user is ADMIN
```

That last condition is business/data authorization and should not be delegated entirely to NGINX.

---

14. Important security consideration

Do **not** blindly trust incoming identity headers such as:

```
X-User
X-Email
X-Role
X-User-ID
```

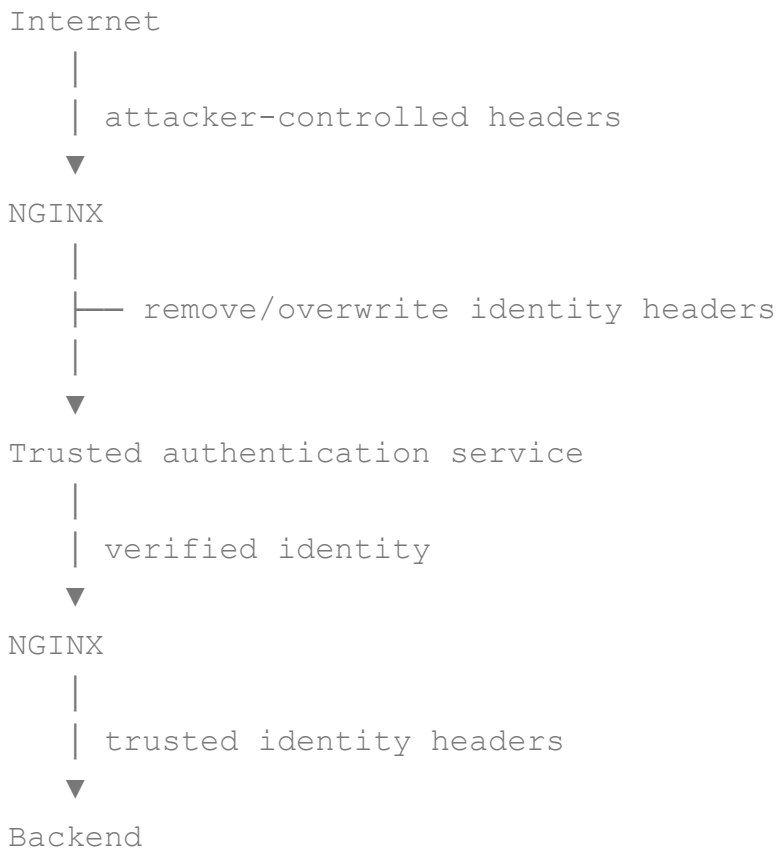
from the Internet.

Otherwise an attacker could send:

```
X-Role: ADMIN
```

and potentially bypass authorization.

The trusted flow should be:



The ingress-nginx external-auth mechanism specifically supports controlled propagation of authentication response headers. Kubernetes

---

15. One important distinction: ingress-nginx vs NGINX Ingress Controller

There are actually two products commonly called "NGINX Ingress":

Community ingress-nginx

Project:

Kubernetes ingress-nginx documentation

Uses annotations such as:

```
nginx.ingress.kubernetes.io/auth-url
nginx.ingress.kubernetes.io/auth-signin
```

F5 NGINX Ingress Controller

Official NGINX/F5 controller:

F5 NGINX Ingress Controller documentation

The newer F5 controller has an `ExternalAuth` Policy CRD, which is a cleaner policy-oriented approach. NGINX introduced `ExternalAuth` in version 5.5.0 and positions it as the migration path from the generic `auth-url` / `auth-signin` pattern. NGINX for Kubernetes

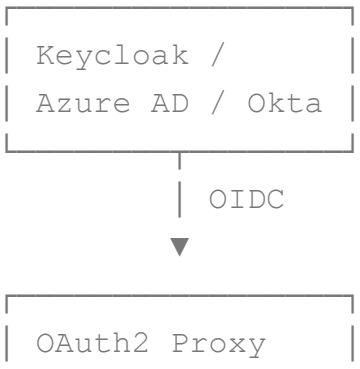
For example:

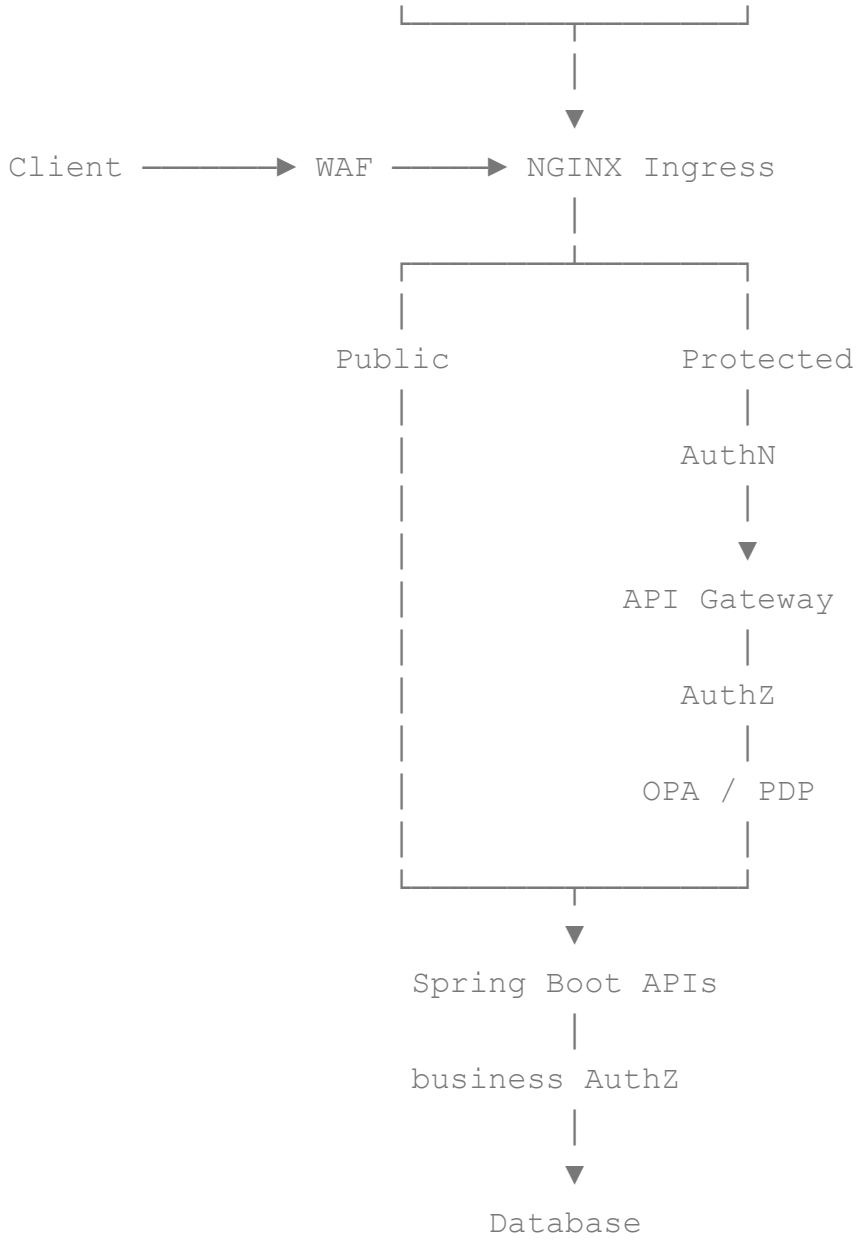
```
apiVersion: k8s.nginx.org/v1
kind: Policy
metadata:
  name: oauth2-auth
spec:
  externalAuth:
    authURI: "/oauth2/auth"
    authSigninURI: "/oauth2/signin"
    authServiceName: "security/oauth2-proxy"
```

That distinction matters when designing your platform because the configuration model is different.

What I would recommend for your architecture

Given your **Spring Boot + Kubernetes + microservices + cloud-native + distributed systems** focus, I would build the security architecture as:





And use:

OIDC → identity
OAuth2 Proxy → browser/session authentication
NGINX → authentication enforcement + routing
OPA/PDP → centralized authorization policy where appropriate
Spring Security → application/business authorization
mTLS/service mesh → east-west workload identity
WAF → edge protection

That gives you a much cleaner separation of concerns than trying to make NGINX itself become the complete IAM/authorization system.

One additional 2026 consideration: if you're starting a **new** platform, first establish whether you mean the community `ingress-nginx` project or the F5 NGINX Ingress Controller. The latter's newer

`ExternalAuth` Policy model is worth evaluating before committing to a large annotation-based design. NGINX for Kubernetes